

그럼 이제 이 프로젝트를 진행하기 전에 다른 팀원들에게
지금까지 나눴던 대화들을 기반으로 이 프로젝트에 대한 총정리
pdf를 제작해서 설명해주려고 해
모든 내용을 다 넣는 건 아니지만 중요한 내용들은 무조건 다
pdf안에 설명이 되어있어야 해 pdf 파일을 생성해줘

팀원분들께 프로젝트의 기획 의도부터 하드웨어 설계 흐름까지 완벽하게 브리핑하실 수
있도록, 지금까지의 모든 핵심 논리와 아키텍처를 담은 [프로젝트 총정리 브리핑 문서]를
마크다운(PDF 변환용) 포맷으로 작성해 드립니다.

브라우저의 인쇄 기능(Ctrl+P 또는 Cmd+P)을 사용해 바로 PDF로 저장하여 팀원들에게
배포하실 수 있습니다.

Zynq SoC 기반 Plasma Etch OES Virtual Metrology NPU 프로젝트 총정리

1. 프로젝트 개요 및 목적

문제 정의: 반도체 플라즈마 식각(Etching) 공정이 끝난 후, 웨이퍼가 얼마나
깎였는지(Etch Depth) 확인하려면 별도의 계측 장비로 이동해야 하며 이는 엄청난
시간과 비용의 낭비(병목)를 초래합니다.

해결책 (Virtual Metrology): 공정 중 챔버 안에서 발생하는 **플라즈마 빛(OES,
Optical Emission Spectroscopy) 데이터**를 실시간으로 수집하여, 계측 장비에 넣기
전에 식각 깊이를 AI로 예측해 냅니다.

최종 목표: 이 AI 연산을 ARM CPU가 아닌, **FPGA 기반의 초고속/초저전력 INT8 CNN
NPU 하드웨어로 가속**하는 Zynq SoC 시스템을 설계 및 구현하는 것입니다.

2. 데이터셋 및 전처리 파이프라인 (SW 파트)

우리는 실제 공개된 BOSCH Plasma Etch 데이터셋(75장 웨이퍼, 8개 Lot)을 사용합니다.
방대한 빛 데이터를 NPU에 넣기 위해 치밀한 공학적 압축을 진행했습니다.

① 왜 'Gas 5' 구간만 사용하는가?

BOSCH 공정은 실리콘을 깎는(Etch) 단계와 벽을 보호하는(Passivation) 단계를
반복합니다.

보호막을 치는 구간(Gas 4)은 깊이와 직접적인 관련이 없고 노이즈만 유발합니다.
따라서 **실제로 바닥이 깎여나가는 핵심 순간인 'Gas 5 (SF6 추정) 커짐' 구간 100번만 추출했습니다.**

② $14,000 \times 3,648 \rightarrow 100 \times 128$ 압축

세로축: 웨이퍼당 100번의 식각(Gas 5) 사이클

가로축: 3,648개의 파장(Wavelength)을 128개 구간으로 Max-pooling (중요한 빛의 Peak 보존)

최종 입력 형태: $100 \times 128 \times 1$ 형태의 2차원 정수(INT8) 이미지 텐서 (약 12.8 KB)

③ 검증 방법: LOLO (Leave-One-Lot-Out)

같은 날 생산된 웨이퍼(같은 Lot)는 챔버 상태를 공유하므로, 랜덤하게 섞어 학습하면 AI가 답을 외워버리는 데이터 누수(Leakage)가 발생합니다.

이를 막기 위해 8개 Lot 중 1개 Lot 전체를 완전히 빼두고(Test), 나머지 7개로만 학습(Train)하는 가장 엄격한 검증 방식을 채택했습니다.

3. 핵심 AI 알고리즘: 잔여 오차(Residual) CNN

직접 식각 깊이를 맞추려던 첫 번째 CNN은 실패했습니다. 우리는 이를 '수학적 구조' 변경으로 돌파했습니다.

챔버 드리프트(Chamber Drift): 챔버는 웨이퍼가 들어갈수록 상태가 변합니다.
웨이퍼 번호만으로 예측한 기본 식각 깊이(Baseline) 오차가 $0.1541 \mu\text{m}$ 였습니다.

Residual(잔여 오차) 보정: NPU는 $40.xx \mu\text{m}$ 전체를 생파로 예측하지 않습니다.

1. PS가 웨이퍼 번호로 대략적인 깊이(Baseline)를 먼저 구합니다. (예: $40.20 \mu\text{m}$)
2. NPU는 OES 빛을 보고 "기준값에서 얼마나 더/덜 깎였는지(Residual)"만 미세하게 예측합니다. (예: $+0.12 \mu\text{m}$)

성능: 이 구조를 도입한 **Residual Tiny CNN**은 오차를 $0.1034 \mu\text{m}$ 까지 획기적으로 줄여냈으며, NPU 탑재를 위해 소수점 연산을 없앤 **INT8 정수 모델로 변환(Quantization)한 후에도 오차($0.1232 \mu\text{m}$)가 거의 발생하지 않음**을 파이썬 Golden Model로 완벽히 검증했습니다.

4. HW / SW 파티셔닝 구조 (시스템 아키텍처)

이 프로젝트의 핵심은 PS(소프트웨어)와 PL(하드웨어)가 각자 제일 잘하는 일을 나누어 맡는 것입니다.

⚙️ PS (ARM 소프트웨어)의 역할

전처리: 원본 데이터 디코딩 → 100 사이클 추출 → 정규화 → **100×128 INT8** 배열 생성

기본 예측: 웨이퍼 순서를 통해 Baseline 계산

결과 복원: NPU가 받은 정수값(INT32)을 받아 물리량(μm)으로 복원 후 최종 깊이 산출

⚙️ PL (NPU 하드웨어)의 역할

가속 연산: PS가 보내준 100×128 이미지와 내부에 고정된 Weight를 가져와, 8개의 MAC 배열로 Conv1 → Pool1 → Conv2 → ... → FC2 연산을 초고속으로 수행.

Requantization: 실수(Float) 곱셈 스케일링을 버리고, 정수 M을 곱한 뒤 우측으로 24칸 시프트(>> 24)하는 방식으로 하드웨어 자원을 극도로 절약.

5. 설계 및 개발 워크플로우 (Vivado → Vitis)

이제 수학적 모델링은 끝났습니다. 앞으로는 다음 5단계의 철저한 하드웨어 엔지니어링을 수행합니다.

Plaintext



[1] Vivado RTL 코딩 및 검증 (가장 중요)

- MAC Array, Requant Unit, BRAM, NPU Controller를 Verilog로 작성.
- 파이썬 Golden Model의 출력값과 RTL 시뮬레이션 값이 1비트의 오차도 없이 동일한지(Bit-

[2] IP 패키징 (Create IP)

- 완성된 npu_core 모듈에 AXI4-Lite(제어용) 래퍼를 씌워 하나의 'oes_npu_ip'로 패키징.

[3] Vivado Block Design (SoC 통합)

- Zynq PS(ARM) → [AXI BRAM Controller] → Input BRAM → NPU IP 연결.
- Weight/Bias는 데이터 전송 병목을 줄이기 위해 NPU 내부 ROM(.mem)에 초기값으로 고정.

[4] 하드웨어 생성 및 XSA 추출

- Synthesis → Implementation → Generate Bitstream → XSA Export.
- 이 과정에서 속도(WNS)와 리소스(LUT, DSP 등) 최적화 확인.

[5] Vitis (ARM C 프로그램 작성)

- PS에서 Input BRAM에 100x128 배열 쓰기 → NPU START → DONE 대기 → RESULT 읽기.
- (Baseline + NPU 예측 오차)를 합산하여 최종 식각 깊이를 UART 시리얼로 출력.

6. 팀원 역할 분담 및 당면 과제

현재 "AI 알고리즘 설계 및 검증" 단계를 완전히 돌파하고, "**디지털 하드웨어 구현**" 단계에 진입했습니다.

A (NPU Core HW): 공유형 8-MAC Array, INT32 누산기, Requantization, Pooling 등 핵심 연산 데이터패스(Datapath) Verilog RTL 코딩.

B (SoC & Control): BRAM 메모리 맵 구성, 어드레스 제너레이터(Address Generator), Layer FSM 제어 로직, AXI4-Lite 연동.

C (SW & Verification): 파이썬 Golden Model C-header 변환, Vitis 기반 ARM 제어 소프트웨어(main.c) 작성, 성능 측정.

당장 내일부터 진행할 첫 번째 마일스톤: